Lab 08
Data Structures
BS DS/CY Fall 2025 Morning/Afternoon

Objective:
Learning and Implementing the Representation of Binary Trees using Array base and implementing some basic operations on them.

ADT for Array Representation

Note: you can add some utility functions as per requirement for the completion of public functions.

```
template<class T>
class BinaryTree
{
private:
        int height;             //represents the maximum possible nodes in (capacity = 2^height -1) tree.
        char *nodeStatus;       //It is used to find that whether there is a node on a particular index of  array.
                                // Note: we are not using the approach of using a sentinel value in 'data' //array because in a template based data T could be of any type.

        T *data;                // stores the nodes of the trees
};
```

Add the following public methods in the Binary Tree Class;

1. BinaryTree(int h); // initializes nodeStatus array with 0 and creates data array of size 2^h -1

2. void setRoot(T v); //stores v at data[0] as root of tree and also sets the nodeStatus[0] =1.

3. T getRoot(); //returns the root of tree if exists.

4. void setLeftChild(T parent, T child);

5. void setRightChild(T parent, T child);

6. T getParent(T node);

7. void remove(T node); //removes the given node and all its descendents from tree.

8. void displayAncestors(T node); //display ancestors of the given node

9. void displayDescendents(T node); //display descendents of the given node

10. void heightOfTree(); //returns the height (actual height) of tree.

11. void preOrder(); // do the VLR of tree.

12. void postOrder(); // do the LRV of tree.

13. void inOrder(); // do the LVR of tree.

14. void levelOrder(); // do the level order traversal of tree.

15. void displayLevel(int levelNo); // display the nodes on a particular level number.

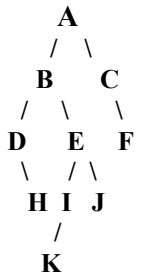
16. int findLevelOfNode(T node); // returns the level/depth of given node.

17. void displayParenthesizedView(); // display the tree in Parenthesize form.

/*

For Example the parenthesize view of the following binary tree will be

A (B (D (, H) E (I (K ,) J)) C (, F))



*/

18. bool isEmpty(); // returns true if the tree has no root (nodeStatus[0] == 0).

19. T getLeftChild(T parent); // returns the left child (index 2i+1) if it exists.

20. T getRightChild(T parent); // returns the right child (index 2i+2) if it exists.

21. bool isLeaf(T node); // returns true if the node has no left and no right child.

22. T getSibling(T node); // returns the other child of the same parent. e.g. getSibling(D) → E, getSibling(H) → no sibling.

23. void updateNode(T oldVal, T newVal); // changes the value of a node in place.

24. ~BinaryTree(); // destructor: frees the data and nodeStatus arrays.

25. int countNodes(); // returns total number of nodes. Expected: 10

26. int countLeaves(); // returns number of leaf nodes. Expected: 4 (H, K, J, F)

27. int countInternalNodes(); // returns number of non-leaf nodes. Expected: 6

28. int countNodesWithOneChild(); // returns number of nodes having exactly one child. Expected: 3 (C, D, I)

29. void displayLeaves(); // displays leaf nodes from left to right. Expected: H K J F

30. void reverseLevelOrder(); // displays levels from bottom to top. Expected: K H I J D E F B C A

31. void spiralOrder(); // zig-zag level order traversal. Expected: A C B D E F J I H K

void displayPath(T node); // displays the path from root to the given node. e.g. displayPath(K) → A B E I K